ADMIN.PY

```
from django.contrib import admin
from django.contrib.auth.admin import UserAdmin as BaseUserAdmin
from .models import User

@admin.register(User)
class UserAdmin(BaseUserAdmin):
    list_display = ('email', 'is_staff', 'is_active')
    list_filter = ('is_staff', 'is_active')

    fieldsets = (
        (None, {'fields': ('email', 'password')}),
        ('Permissions', {'fields': ('is_staff', 'is_active', 'is_superuser')}),
    )

    add_fieldsets = (
        (None, {
            'fields': ('email', 'password1', 'password2', 'is_staff', 'is_active'),
        }),
    )

    search_fields = ('email',)
    ordering = ('email',)
```

APPS.PY

```
from django.apps import AppConfig

class UserConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'user'
```

BACKENDS.PY

```
from django.contrib.auth.backends import ModelBackend
from django.contrib.auth import get_user_model

class EmailBackend(ModelBackend):
    def authenticate(self, request, username=None, password=None, **kwargs):
        UserModel = get_user_model()

        try:
            user = UserModel.objects.get(email=username)
            if user.check_password(password):
                return user
        except UserModel.DoesNotExist:
            return None

        return None
```

MODELS.PY

```
from django.contrib.auth.models import AbstractBaseUser, BaseUserManager, PermissionsMixin
from django.db import models
from django.utils import timezone

class UserManager(BaseUserManager):
    def create_user(self, email, password=None, **extra_fields):
        if not email:
            raise ValueError('Email is required')

        email = self.normalize_email(email)
        user = self.model(email=email, **extra_fields)
        if password:
            user.set_password(password)
        user.save(using=self._db)
```

```
        return user

    def create_superuser(self, email, password=None, **extra_fields):
        extra_fields.setdefault('is_staff', True)
        extra_fields.setdefault('is_superuser', True)
        return self.create_user(email, password, **extra_fields)

class User(AbstractBaseUser, PermissionsMixin):
    email = models.EmailField(unique=True)
    first_name = models.CharField(max_length=100, blank=True, default='')
    last_name = models.CharField(max_length=100, blank=True, default='')
    phone_number = models.CharField(max_length=15, blank=True, default='')

    # Tenant and role fields
    tenant = models.IntegerField(null=True, blank=True)
    role = models.CharField(max_length=50, blank=True, default='')
    role_id = models.IntegerField(null=True, blank=True)

    # Supervisor fields
    supervisor_name = models.CharField(max_length=200, blank=True, default='')
    supervisor_email = models.EmailField(blank=True, default='')
    supervisor_mobile = models.CharField(max_length=15, blank=True, default='')

    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)
    date_joined = models.DateTimeField(default=timezone.now)  # Changed from auto_now_add

    objects = UserManager()

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = []
```

```
    class Meta:
        db_table = 'users'

    def __str__(self):
        return self.email
```

PERMISSIONS.PY

```
from rest_framework import permissions


class DefaultPermission(permissions.BasePermission):
    """
    Default permission class - allows authenticated users
    """
    def has_permission(self, request, view):
        return request.user and request.user.is_authenticated


class IsAdminUser(permissions.BasePermission):
    """
    Permission for admin users only
    """
    def has_permission(self, request, view):
        return request.user and request.user.is_authenticated and request.user.is_staff


class IsOwnerOrReadOnly(permissions.BasePermission):
    """
    Custom permission to only allow owners of an object to edit it.
    """
    def has_object_permission(self, request, view, obj):
        # Read permissions are allowed to any request
        if request.method in permissions.SAFE_METHODS:
```

```
        return True

    # Write permissions are only allowed to the owner
    return obj.user == request.user
```

SERIALIZERS.PY

```
from rest_framework import serializers
from django.contrib.auth import get_user_model
from django.contrib.auth.password_validation import validate_password

User = get_user_model()

class UserSerializer(serializers.ModelSerializer):
    """
    Serializer for user model - handles all user fields
    """
    # Use source to map frontend fields to backend fields
    first_name = serializers.CharField(required=False, allow_blank=True)
    last_name = serializers.CharField(required=False, allow_blank=True)
    phone = serializers.CharField(required=False, allow_blank=True, source='phone_number')
    mobile_no = serializers.CharField(required=False, allow_blank=True, source='phone_number',
write_only=True)

    # Additional fields from your frontend
    tenant = serializers.IntegerField(required=False, allow_null=True)
    tenant_id = serializers.IntegerField(required=False, allow_null=True, source='tenant')
    role = serializers.CharField(required=False, allow_blank=True)
    role_type = serializers.CharField(required=False, allow_blank=True, source='role')
    role_id = serializers.IntegerField(required=False, allow_null=True)

    # Supervisor fields
    supervisor_name = serializers.CharField(required=False, allow_blank=True)
```

```
    supervisor_email = serializers.EmailField(required=False, allow_blank=True)
    supervisor_mobile = serializers.CharField(required=False, allow_blank=True)

    password = serializers.CharField(write_only=True, required=False, allow_blank=True)

    class Meta:
        model = User
        fields = (
            'id', 'email', 'first_name', 'last_name', 'phone', 'mobile_no',
            'tenant', 'tenant_id', 'role', 'role_type', 'role_id',
            'supervisor_name', 'supervisor_email', 'supervisor_mobile',
            'is_active', 'password'
        )
        read_only_fields = ('id',)
        extra_kwargs = {
            'password': {'write_only': True}
        }
```

URLS.PY

```
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import LoginView, LogoutView, UserViewSet

router = DefaultRouter()
router.register('users', UserViewSet, basename='users')

urlpatterns = [
    path('login/', LoginView.as_view(), name='login'),
    path('logout/', LogoutView.as_view(), name='logout'),
    path('', include(router.urls)),
]
```

VIEWS.PY

```
from django.contrib.auth import authenticate, login, logout
from django.shortcuts import render, redirect
from django.contrib import messages
from django.views import View
from rest_framework import viewsets, permissions, status
from rest_framework.response import Response
from django.contrib.auth import get_user_model
from .serializers import UserSerializer

User = get_user_model()

class UserViewSet(viewsets.ModelViewSet):
    """
    ViewSet for CRUD operations on users
    """
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [permissions.IsAuthenticated]

    def create(self, request, *args, **kwargs):
        """Create a new user"""
        data = request.data.copy()

        # Extract password if provided
        password = data.pop('password', None)

        # Create user instance
        serializer = self.get_serializer(data=data)
        serializer.is_valid(raise_exception=True)
```

```
        user = serializer.save()

        # Set password if provided
        if password:
            user.set_password(password)
            user.save()

        headers = self.get_success_headers(serializer.data)
        return Response(serializer.data, status=status.HTTP_201_CREATED, headers=headers)

    def update(self, request, *args, **kwargs):
        """Update user details"""
        partial = kwargs.pop('partial', False)
        instance = self.get_object()
        data = request.data.copy()

        # Extract and handle password separately
        password = data.pop('password', None)

        serializer = self.get_serializer(instance, data=data, partial=partial)
        serializer.is_valid(raise_exception=True)
        self.perform_update(serializer)

        # Update password if provided and not empty
        if password:
            instance.set_password(password)
            instance.save()

        return Response(serializer.data)
```

```
class LoginView(View):
    def get(self, request):
        return render(request, 'user/login.html')

    def post(self, request):
        email = request.POST.get('email')
        password = request.POST.get('password')

        user = authenticate(request, username=email, password=password)

        if user is not None:
            login(request, user)
            messages.success(request, 'Login successful!')
            return redirect('home')
        else:
            messages.error(request, 'Invalid email or password')

        return render(request, 'user/login.html')

class LogoutView(View):
    def get(self, request):
        logout(request)
        messages.success(request, 'Logged out successfully!')
        return redirect('login')

    def post(self, request):
        logout(request)
        messages.success(request, 'Logged out successfully!')
        return redirect('login')

def login_view(request):
```

```
    if request.method == 'POST':
        email = request.POST.get('email')
        password = request.POST.get('password')

        user = authenticate(request, username=email, password=password)

        if user is not None:
            login(request, user)
            messages.success(request, 'Login successful!')
            return redirect('home')
        else:
            messages.error(request, 'Invalid email or password')

    return render(request, 'user/login.html')

def logout_view(request):
    logout(request)
    messages.success(request, 'Logged out successfully!')
    return redirect('login')
```